

Paper 3 — Tactile Data Engine & Learned Contact Extrapolation

End-to-end pipeline: collect synthetic tactile data in simulation, stitch it into extended-contact ground truth, train a model to predict the full contact from a single initial touch, and validate it against unseen simulated and real objects.

Core idea. Paper 1 classified an object's shape from the initial tactile imprint and applied hand-coded rules to guess the unseen contact. Paper 3 replaces that with a **learned** model trained on large-scale simulation data: input the initial contact, output the extended contact map. The shape classifier disappears; the evaluation metrics (TC, GSR, SSIM) and the regrasp pipeline are reused.

Block 1 — Data collection

the part we are building now

main.py — orchestrator (PyCharm)

One controller script you run in PyCharm. You enter the object, its pose, the grid of contact points, the step size, and any rotation angles. It runs the preview, then hands off collection to the simulator, then plots the result.

Why: One place to drive everything, so the workflow is repeatable and you don't edit numbers in several files.

Object manager — STL→USD (added later)

Converts any STL shape into the USD format the simulator loads, so new objects drop into the same scene as the cylinder.

Why: Paper 3 needs many shapes; this makes swapping objects a one-step action instead of manual work.

scene_config.py — poses, grid, preview (PyCharm)

Holds the object pose and the pad grid in plain numbers, and draws a to-scale sketch (map, front, side) of how the pad meets the object. Saves the grid as CSV and a JSON scene record.

Why: Lets you decide a good grid cheaply, by eye, before spending minutes on a real simulation run.

Isaac Sim collect — terminal, per grid point

Runs the simulator once per grid point. The gripper approaches, closes, the tactile sensors record the pressure, then it opens and moves to the next point. Writes one CSV set per point.

Why: Must run in the terminal with Isaac's own Python — it cannot run inside PyCharm. This is the one hard boundary in the pipeline.

Scene record — JSON for lab replay

Every run saves a JSON with the object pose, robot base, grid, and settings, copied next to the data.

Why: So the exact same scene and motion can be recreated on the real UR5e at the CoRo lab later — essential for sim-to-real comparison.

Tactile CSVs + heatmaps

Each grasp produces the raw tactile time-series; the plotter reduces each to one map (hold-averaged) and lays the grid of heatmaps out in position.

Why: Confirms the contact shifts sensibly across the grid — the signal the model will learn from.

Block 2 — Data preparation

build the training pairs

Grid of contacts → stitch / mosaic → extended map

The many small contact maps from one object are assembled into a single large extended-contact map — the full picture of how that object contacts the pad across the scanned region.

Why: This large map is the ground truth the model must learn to predict.

Center contact (input) + extended map (target) → dataset

Each training example pairs the single center-contact map (input) with the stitched extended map (target). Repeated across many objects and poses, this becomes the dataset.

Why: Defines the exact learning task: from one initial touch, reconstruct the whole contact — Paper 1's goal, now learned.

Block 3 — Training and validation

two models, head to head

Model A — simulation data only

A CNN / U-Net trained purely on simulated tactile data.

Why: Tests how far cheap, abundant simulation data alone can take the model.

Model B — simulation + a dose of real data

The same architecture trained on simulation plus a small amount of real CoRo-lab tactile data.

Why: Vincent's suggestion: see whether a little real data meaningfully closes the sim-to-real gap.

Validate on unseen objects — sim + real, Paper 1 metrics

Both models are tested on objects they never saw, in simulation and on the real robot, scored with Tactile Centroid error, Grasp Success Rate error, and SSIM.

Why: Reusing Paper 1's metrics lets you compare old (rules) vs new (learned) on equal footing, and proves the model generalizes.

Block 4 — Result

the headline contribution

Compare A vs B — measure the sim-to-real benefit

Quantify how much the dose of real data improved performance over the sim-only model.

Why: Answers the central question — how much real data is worth — and gives the paper a concrete, publishable finding alongside the learned-extrapolation contribution.

Where we are now: bottom of Block 1 — verifying that the collected tactile heatmaps match the preview on a small 3×3 grid. Everything downstream depends on this data being trustworthy, which is why the small loop is proven before building the stitching (Block 2).

Combined pipeline flowchart

The four blocks at a glance — from data collection through the final sim-to-real comparison.

